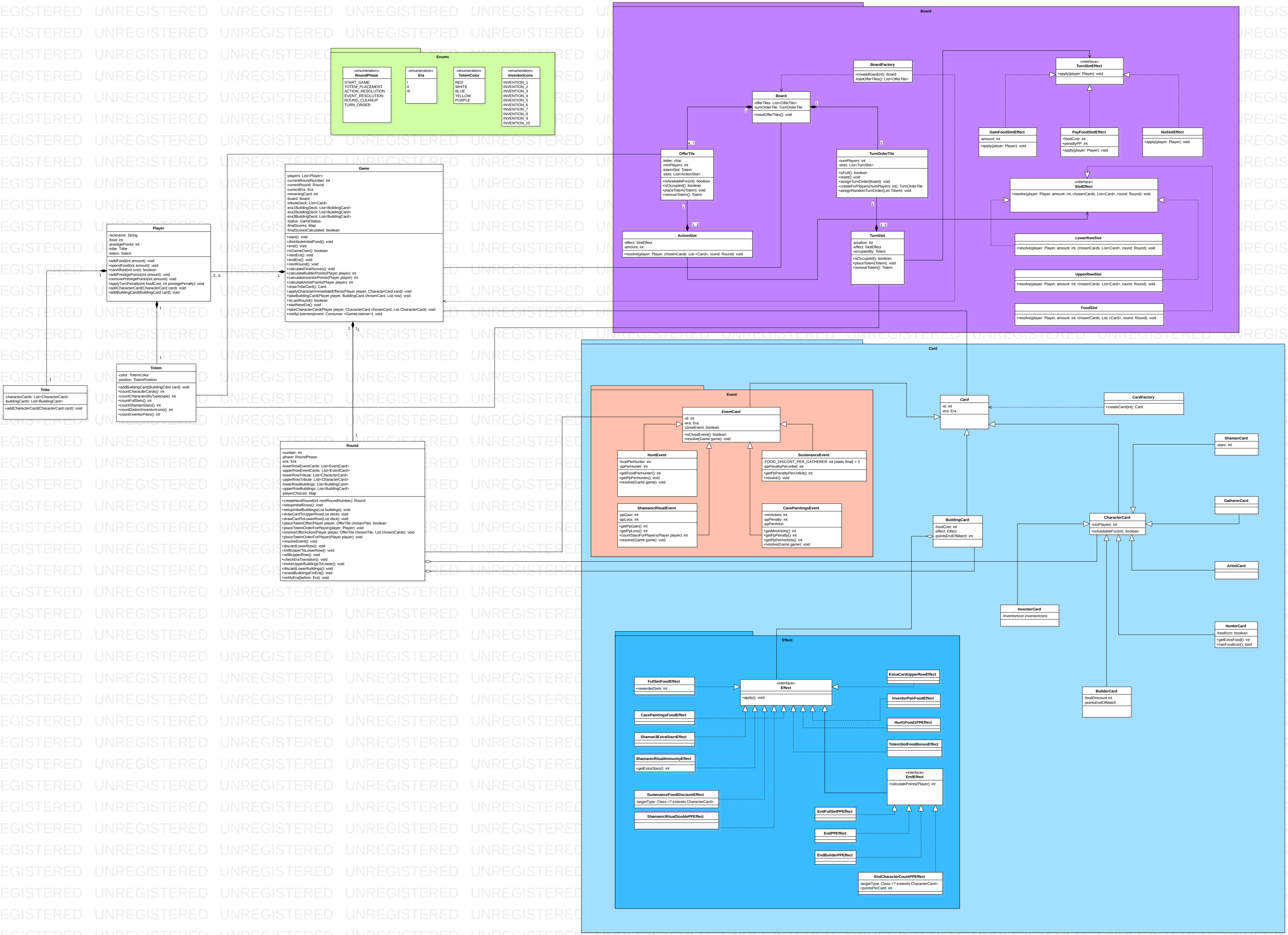
UNREGISTERED

Model::Main


Enums
«enumeration»
RoundPhase
START_GAME
TOTEM_PLACEMENT
ACTION_RESOLUTION
EVENT_RESOLUTION
ROUND_CLEANUP
TURN_ORDER
«enumeration»
Era
I
II
III
«enumeration»
TotemColor
RED
WHITE
BLUE
YELLOW
PURPLE
«enumeration»
InventorIcons
INVENTION_1
INVENTION_2
INVENTION_3
INVENTION_4
INVENTION_5
INVENTION_6
INVENTION_7
INVENTION_8
INVENTION_9
INVENTION_10
Game
-players: List<Player>
-currentRoundNumber: int
-currentRound: Round
-currentEra: Era
-remaningCard: int
-board: Board
-tributeDeck: List<Card>
-era1BuildingDeck: List<BuildingCard>
-era2BuildingDeck: List<BuildingCard>
-era3BuildingDeck: List<BuildingCard>
-status: GameStatus
-finalScores: Map
-finalScoresCalculated: boolean
+start(): void
+distributeInitialFood(): void
+end(): void
+isGameOver(): boolean
+nextEra(): void
+endEra(): void
+nextRound(): void
+calculateFinalScores(): void
+calculateBuilderPoints(Player player): int
+calculateInventorPoints(Player player): int
+calculateArtistPoints(Player player): int
+drawTribeCard(): Card
+applyCharacterImmediateEffects(Player player, CharacterCard card): void
+takeBuildingCard(Player player, BuildingCard chosenCard, List row): void
+isLastRound(): boolean
+startNewEra(): void
+takeCharacterCard(Player player, CharacterCard chosenCard, List CharacterCard): void
+notifyListeners(event: Consumer <GameListener>): void
1
1
1
1
Player
-nickname: String
-food: int
-prestigePoints: int
-tribe: Tribe
-totem: Totem
+addFood(int amount): void
+spendFood(int amount): void
+canAfford(int cost): boolean
+addPrestigePoints(int amount): void
+removePrestigePoints(int amount): void
+applyTurnPenalty(int foodCost, int prestigePenalty): void
+addCharacterCard(CharacterCard card): void
+addBuildingCard(BuildingCard card): void
1
2...5
1
1
1
Tribe
-characterCards: List<CharacterCard>
-buildingCards: List<BuildingCard>
+addCharacterCard(CharacterCard card): void
Totem
-color: TotemColor
-positon: TotemPosition
+addBuildingCard(BuildingCard card): void
+countCharacterCards(): int
+countCharactersByType(type): int
+countFullSets(): int
+countShamanStars(): int
+countDistinctInventorIcons(): int
+countInventorPairs(): int
1
Round
-number: int
-phase: RoundPhase
-era: Era
-lowerRowEventCards: List<EventCard>
-upperRowEventCards: List<EventCard>
-lowerRowTribute: List<CharacterCard>
-upperRowTribute: List<CharacterCard>
-lowerRowBuildings: List<BuildingCard>
-upperRowBuildings: List<BuildingCard>
-playerChoices: Map
+createNextRound(int nextRoundNumber): Round
+setupInitialRows(): void
+setupInitialBuildings(List buildings): void
+drawCardToUpperRow(List deck): void
+drawCardToLowerRow(List deck): void
+placeTotemOffer(Player player, OfferTile chosenTile): boolean
+placeTotemOrderForPlayer(player: Player): void
+resolveOfferAction(Player player, OfferTile chosenTile, List chosenCards): void
+placeTotemOrderForPlayer(Player player): void
+resolveEvent(): void
+discardLowerRow(): void
+shiftUpperToLowerRow(): void
+refillUpperRow(): void
+checkEraTransition(): void
+moveUpperBuildingsToLower(): void
+discardLowerBuildings(): void
+revealBuildingsForEra(): void
+notifyEra(before: Era): void
Board
BoardFactory
+createBoard(int): Board
-loadOfferTiles(): List<OfferTile>
Board
-offerTiles: List<OfferTile>
-turnOrderTile: TurnOrderTile
+resetOfferTiles(): void
1
1
4..7
1
OfferTile
-letter: char
-minPlayers: int
-totemSlot: Totem
-slots: List<ActionSlot>
+isAvailableFor(int): boolean
+isOccupied(): boolean
+placeTotem(Totem): void
+removeTotem(): Totem
TurnOrderTile
-numPlayers: int
-slots: List<TurnSlot>
+isFull(): boolean
+reset(): void
+assignTurnOrder(Board): void
+createForPlayers(numPlayers: int): TurnOrderTile
+assignRandomTurnOrder(List Totem): void
1
1..3
1
2..5
ActionSlot
-effect: SlotEffect
-amount: int
+resolve(player: Player, chosenCards: List <Card>, round: Round): void
TurnSlot
-position: int
-effect: SlotEffect
-occupiedBy: Totem
+isOccupied(): boolean
+placeTotem(Totem): void
+removeTotem(): Totem
«interface»
TurnSlotEffect
+apply(player: Player): void
GainFoodSlotEffect
-amount: int
+apply(player: Player): void
PayFoodSlotEffect
+foodCost: int
+penaltyPP: int
+apply(player: Player): void
NoSlotEffect
+apply(player: Player): void
«interface»
SlotEffect
+resolve(player: Player, amount: int, chosenCards: List<Card>, round: Round): void
LowerRowSlot
+resolve(player: Player, amount: int, chosenCards: List<Card>, round: Round): void
UpperRowSlot
+resolve(player: Player, amount: int, chosenCards: List<Card>, round: Round): void
FoodSlot
+resolve(player: Player, amount: int, chosenCards: List <Card>, round: Round): void
Card
Event
EventCard
-id: int
-era: Era
-closeEvent: boolean
+isCloseEvent(): boolean
+resolve(Game game): void
HuntEvent
-foodPerHunter: int
-ppPerHunter: int
+getFoodPerHunter(): int
+getPpPerHunter(): void
+resolve(Game game): void
SustenanceEvent
-FOOD_DISCONT_PER_GATHERER: int {static final} = 3
-ppPenaltyPerUnfed: int
+getPpPenaltyPerUnfed(): int
+resolve(): void
ShamanicRitualEvent
-ppGain: int
-ppLoss: int
+getPpGain(): int
+getPpLoss(): int
+countStarsForPlayers(Player player): int
+resolve(Game game): void
CavePaintingsEvent
-minArtists: int
-ppPenalty: int
-ppPerArtist
+getMinArtists(): int
+getPpPenatly(): int
+getPpPerAsrtists(): int
+resolve(Game game): void
Card
-id: int
-era: Era
CardFactory
+createCard(int): Card
ShamanCard
-stars: int
GathererCard
BuildingCard
-foodCost: int
-effect: Effect
-pointsEndOfMatch: int
CharacterCard
-minPlayers: int
+isAvailableFor(int): boolean
ArtistCard
InventorCard
-InventorIcon inventorIcons
HunterCard
-foodIcon: boolean
+getExtraFood(): int
+hasFoodIcon(): bool
BuilderCard
-foodDiscount int
-pointsEndOfMatch
Effect
FullSetFoodEffect
+rewardedSets: int
«interface»
Effect
+apply(): void
ExtraCardUpperRowEffect
InventorPairFoodEffect
CavePaintingsFoodEffect
Hunt1Food1PPEffect
Shaman3ExtraStarsEffect
TotemSlotFoodBonusEffect
ShamanicRitualImmunityEffect
+getExtraStars(): int
«interface»
EndEffect
+calculatePoints(Player): int
SustenanceFoodDiscountEffect
-targetType: Class <? extends CharacterCard>
ShamanicRitualDoublePPEffect
EndFullSetPPEffect
EndPPEffect
EndBuilderPPEffect
EndCharacterCountPPEffect
-targetType: Class <? extends CharacterCard>
+pointsPerCard: int

Model1::ClassDiagram1

UNREGISTERED